

The Relational Solver of cvc5

A guided tour of `src/theory/sets/theory_sets_rels.{h,cpp}`

algorithms, data structures, invariants,
and an investigation of the transitive-closure graph construction

Abstract

This document explains, for a reader who has never opened the file before, what `theory_sets_rels.cpp` does and why. It is organised in four parts. Part I recalls the theory of finite relations $T_{\mathcal{R}}$ and the tableaux calculus of Meng, Reynolds, Tinelli and Barrett (CADE 2017) [1], because every function in the file is a direct transcription of one of that paper’s derivation rules. Part II documents the data structures declared in the header, the invariants each of them is supposed to satisfy, and their lifetimes. Part III walks through the algorithms in pseudocode, rule by rule, culminating in a detailed treatment of transitive closure — by far the most intricate part of the file. Part IV answers a concrete engineering question: *should `buildTCGraphForRel` be called once, up front, right after `collectRelsInfo`, or twice under guards, as `main` does?* The answer is *once*. A second call rebuilds the graph from the base relation’s members alone, so it discards the closure edges that the first call’s rule applications have added in the meantime — which is why `main` has to guard both call sites to stay correct. Building once makes the guards unnecessary, and turns Invariant 5 into a property that holds by construction rather than by iteration order. The two configurations are behaviourally and performance-indistinguishable (Section 15): no answer and no inference count differs on 167 benchmarks, and 900 fuzzed formulas agree with an exact reachability oracle in both. So this is a structural argument, not a speedup; a patch, the benchmark family that exercises the interaction, the differential fuzzer and the full measurement are included.

Baselines. `main` is 5cc03f4b9, which contains PR #12793 (c851f83bf, the commit that added the guards). The implementation is commit 8bcd3239 on branch `build-trc-graph-once`; line numbers throughout refer to it unless stated otherwise.

Contents

I	The theory and the calculus	3
1	Why this file exists	3
2	The calculus in one page	3
3	Where the file sits in cvc5	4
II	Data structures	5
4	TupleTrie: an index of the members of a relation	5
5	The fields of <code>TheorySetsRels</code>	6
5.1	Two maps that are easy to confuse	7
III	The algorithms	7

6	The top level: <code>check</code>	7
7	<code>collectRelsInfo</code>: rebuilding the world	9
8	Explanations: the discipline that makes the rest work	10
9	The non-closure rules	11
9.1	Transpose	11
9.2	Product and join, downward	12
9.3	Product and join, upward: <code>composeMembersForRels</code>	12
9.4	Table join	12
9.5	Identity	13
9.6	Join image	13
10	Transitive closure in detail	13
10.1	The idea	13
10.2	<code>buildTCGraphForRel</code> : materialising the base graph	14
10.3	<code>applyTCRule</code> : TCLOS DOWN plus graph augmentation	14
10.4	<code>isTCReachable</code> : the entailment guard	15
10.5	<code>doTCInference</code> : enumerating paths	16
10.6	The transitive-closure invariants	17
IV	Should <code>buildTCGraphForRel</code> be called once or twice?	17
11	The question	17
12	Static analysis	18
13	The benchmarks that exercise the interaction	19
13.1	The in-tree regression that used to be disabled	20
14	The patch	20
14.1	Why not keep the guards?	21
15	Measurement	22
15.1	Setup	22
15.2	Answers and inference counts	22
15.3	Scaling families	22
15.4	The one case where once could do more work	23
15.5	Differential fuzzing against an exact oracle	23
16	Conclusion of the investigation	24
17	Other rough edges noticed along the way	24
A	Quick reference: file map	25
B	Reproducing the experiment	26

Part I

The theory and the calculus

1 Why this file exists

In `cvc5` a *relation* is not a primitive object. A relation of arity n is a *set of n -tuples*:

$$\text{Rel}_n(\sigma_1, \dots, \sigma_n) \equiv \text{Set}(\text{Tup}_n(\sigma_1, \dots, \sigma_n)).$$

Everything that is true of sets is therefore already true of relations, and the existing theory of finite sets T_S [2] handles union, intersection, difference, singletons and membership for free. What T_S does *not* know is that some set terms are built with *relational* operators: transpose, product, join, transitive closure, identity, join image. Supplying the reasoning for exactly those operators is the entire job of `TheorySetsRels`, and this is why the class is described in its own header as “the relations extension of the theory of sets” and is owned by `TheorySetsPrivate` rather than being a theory of its own.

The signature $\Sigma_{\mathcal{R}}$ of the theory, reproduced from Figure 1 of [1], is:

<i>Set symbols</i> (inherited from T_S)	
$\emptyset : \text{Set}(\alpha)$	$\sqcup, \sqcap, \setminus : \text{Set}(\alpha) \times \text{Set}(\alpha) \rightarrow \text{Set}(\alpha)$
$\{_ \} : \alpha \rightarrow \text{Set}(\alpha)$	$\in : \alpha \times \text{Set}(\alpha) \rightarrow \text{Bool}, \quad \sqsubseteq : \text{Set}(\alpha) \times \text{Set}(\alpha) \rightarrow \text{Bool}$
<i>Relation symbols</i>	
$\langle _, \dots, _ \rangle : \alpha_1 \times \dots \times \alpha_n \rightarrow \text{Tup}_n(\alpha_1, \dots, \alpha_n)$	tuple constructor
$\times : \text{Rel}_m(\alpha) \times \text{Rel}_n(\beta) \rightarrow \text{Rel}_{m+n}(\alpha, \beta)$	product
$\bowtie : \text{Rel}_{p+1}(\alpha, \gamma) \times \text{Rel}_{q+1}(\gamma, \beta) \rightarrow \text{Rel}_{p+q}(\alpha, \beta)$	join, $p + q > 0$
$(\cdot)^{-1} : \text{Rel}_m(\alpha_1, \dots, \alpha_m) \rightarrow \text{Rel}_m(\alpha_m, \dots, \alpha_1)$	transpose
$(\cdot)^+ : \text{Rel}_2(\alpha, \alpha) \rightarrow \text{Rel}_2(\alpha, \alpha)$	transitive closure

`cvc5` has since grown three further operators that the 2017 paper does not cover but that live in the same file: `RELATION_IDEN` (the identity relation $\text{iden}(R) = \{\langle x, x \rangle \mid \langle x \rangle \in R\}$), `RELATION_JOIN_IMAGE` ($R_{\text{join}n}$, the set of left elements with at least n distinct right neighbours), and `RELATION_TABLE_JOIN` (`SMT-LIB` (`_ rel.table_join m1 n1 ...`), an equi-join on explicit column indices). They are handled by the same machinery, so Section 9 treats them alongside the paper’s rules.

▷ *Notation.* The paper writes $x @ s$ for membership; this document writes $x \in s$. Tuples are $\langle a, b \rangle$; in `SMT-LIB` syntax the same object is `(tuple a b)`, and internally it is an `APPLY_CONSTRUCTOR` node over the tuple datatype’s single constructor. $TC(r)$ and r^+ both denote `(rel.tclosure r)`.

2 The calculus in one page

The calculus of [1] is a tableau over a state S , which is either `unsat` or a set of $T_{\mathcal{R}}$ -constraints. Rules are read “if the premises are in S^* , extend S ”; S^* is the congruence closure of S extended with tuple reasoning, i.e. the set of equalities, disequalities and memberships entailed by S modulo equality. Rules with $\parallel$ are branching (they become *splitting lemmas* in the implementation).

Every operator gets a `DOWN` rule (given a membership in the compound term, deduce memberships in the arguments) and an `UP` rule (given memberships in the arguments and the compound term occurring in S , deduce a membership in the compound term). This up/down duality is the single most important thing to internalise: it is the skeleton of the whole file.

$$\begin{array}{c}
\text{TRANSP UP} \frac{\langle x_1, \dots, x_n \rangle \in R \quad R^{-1} \in \mathcal{T}(S)}{S := S, \langle x_n, \dots, x_1 \rangle \in R^{-1}} \\
\text{PROD UP} \frac{\langle \vec{x} \rangle \in R_1 \quad \langle \vec{y} \rangle \in R_2 \quad R_1 \times R_2 \in \mathcal{T}(S)}{S := S, \langle \vec{x}, \vec{y} \rangle \in R_1 \times R_2} \\
\text{JOIN UP} \frac{\langle \vec{x}, z \rangle \in R_1 \quad \langle z, \vec{y} \rangle \in R_2 \quad R_1 \bowtie R_2 \in \mathcal{T}(S)}{S := S, \langle \vec{x}, \vec{y} \rangle \in R_1 \bowtie R_2} \\
\text{TCLOS UP I} \frac{\langle x_1, x_2 \rangle \in R \quad R^+ \in \mathcal{T}(S)}{S := S, \langle x_1, x_2 \rangle \in R^+} \\
\text{TCLOS DOWN} \frac{\langle x_1, x_2 \rangle \in R^+}{S := S, \langle x_1, x_2 \rangle \in R \quad \parallel \quad S := S, \langle x_1, z \rangle \in R, \langle z, x_2 \rangle \in R \\ \parallel \quad S := S, \langle x_1, z_1 \rangle \in R, \langle z_1, z_2 \rangle \in R^+, \langle z_2, x_2 \rangle \in R, z_1 \not\approx z_2}
\end{array}
\qquad
\begin{array}{c}
\text{TRANSP DOWN} \frac{\langle x_1, \dots, x_n \rangle \in R^{-1}}{S := S, \langle x_n, \dots, x_1 \rangle \in R} \\
\text{PROD DOWN} \frac{\langle \vec{x}, \vec{y} \rangle \in R_1 \times R_2 \quad |\vec{x}| = \text{ar}(R_1)}{S := S, \langle \vec{x} \rangle \in R_1, \langle \vec{y} \rangle \in R_2} \\
\text{JOIN DOWN} \frac{\langle \vec{x}, \vec{y} \rangle \in R_1 \bowtie R_2 \quad |\vec{x}| + 1 = \text{ar}(R_1)}{S := S, \langle \vec{x}, z \rangle \in R_1, \langle z, \vec{y} \rangle \in R_2} \\
\text{TCLOS UP II} \frac{\langle x_1, x_2 \rangle \in R^+ \quad \langle x_2, x_3 \rangle \in R^+}{S := S, \langle x_1, x_3 \rangle \in R^+}
\end{array}$$

In JOIN DOWN and TCLOS DOWN, z, z_1, z_2 are fresh (Skolem) variables. TCLOS DOWN is the only rule that can be applied indefinitely: its third branch re-introduces a R^+ membership over fresh variables, so a naive strategy diverges. That is why the implementation guards it aggressively (Section 10.3).

► *Correctness properties from [1]: the calculus is refutation sound (Prop. 1) and model sound (Prop. 2) in general, and terminating (Prop. 3) only for the restricted fragment of Figure 4 of the paper — unary and binary relations, no transitive closure, no product, no equality between relations. Everything the implementation does outside that fragment, transitive closure in particular, is best-effort.*

3 Where the file sits in cvc5

`TheorySetsRels::check` is called from `TheorySetsPrivate::fullEffortCheck` (`theory_sets_private.cpp`) as the *last* step of the sets full-effort loop, and only when `d_rels_enabled` is set — i.e. when at least one relational term has been registered. All the pure-set rules (downwards/upwards closure, filter, map, group, disequalities, comprehension reduction, cardinality) run first and, if any of them sent something, the loop restarts and the relational solver is not reached at all.

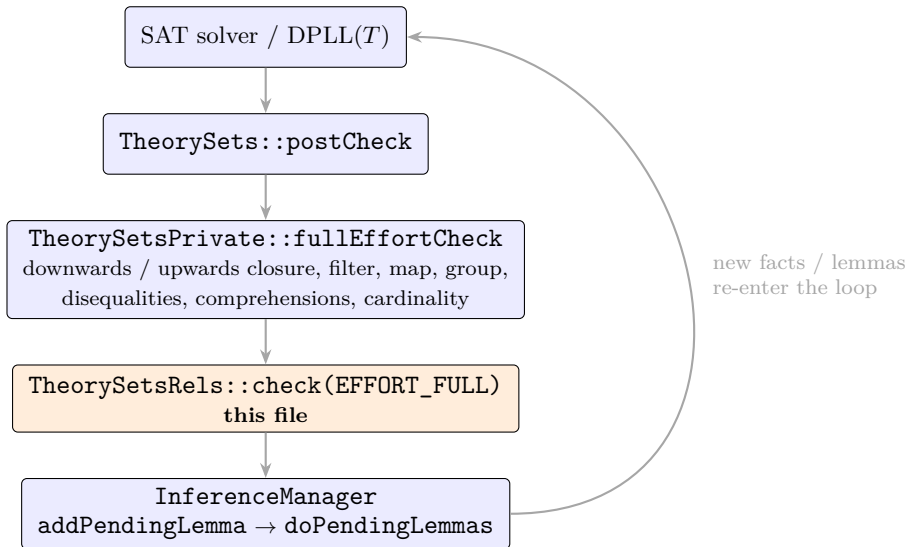


Figure 1: Position of the relational solver in the sets check loop.

Two consequences of this placement matter throughout:

1. **The equality engine is the source of truth.** The relational solver never stores assertions of its own. At the beginning of every full-effort call it rebuilds all of its indices by walking the equality engine (`collectRelsInfo`), and at the end it throws them away. All of the “caches” in the header are therefore *scratch space for one call*, not persistent state.
2. **Nothing is asserted directly.** Conclusions are accumulated as pending lemmas via `sendInfer` and flushed by `d_im.doPendingLemmas()` after `check` returns. A rule application is thus never immediately visible to a later rule in the same call; it becomes visible in the *next* full-effort call, once the SAT solver has propagated it. This is why the file re-derives so much on every call, and why “how many times do we compute this graph per call” is a meaningful question.

Part II

Data structures

4 TupleTrie: an index of the members of a relation

```
class TupleTrie {
public:
    std::map<Node, TupleTrie> d_data;
    std::vector<Node> findTerms(std::vector<Node>& reps, int argIndex = 0);
    std::vector<Node> findSuccessors(std::vector<Node>& reps, int argIndex = 0);
    Node existsTerm(std::vector<Node>& reps, int argIndex = 0);
    bool addTerm(Node n, std::vector<Node>& reps, int argIndex = 0);
};
```

A `TupleTrie` is a prefix tree over the *equivalence-class representatives* of a tuple’s components. For a tuple $t = \langle e_1, \dots, e_n \rangle$ the path is $\text{rep}(e_1) \rightarrow \text{rep}(e_2) \rightarrow \dots \rightarrow \text{rep}(e_n) \rightarrow t$: note the extra final level, which stores the tuple term itself as a key with an empty subtree. That is what the comment “`d_data[n].clear()` ... interpreted as the data and not as a reference to a child” in `addTerm` means.

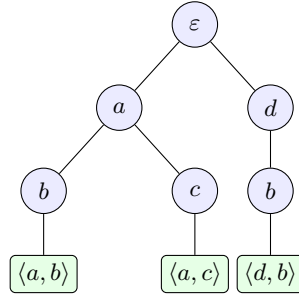


Figure 2: `d_membership_trie[rep(R)]` after asserting $\langle a, b \rangle, \langle a, c \rangle, \langle d, b \rangle \in R$. Round nodes are component representatives; the boxed leaves are the stored tuple terms.

The four operations differ only in where they stop and what they return:

Method	Stops at	Returns
addTerm (n, reps)	level reps	inserts n as the leaf key; false if a tuple with the same representative vector is already present (this is the de-duplication used by collectRelsInfo).
existsTerm (reps)	level reps	the stored tuple, or null : “is there already a member with exactly these component representatives?”
findSuccessors (reps)	level reps	<i>all keys one level below the given prefix</i> . With reps a 1-element prefix [rep(<i>x</i>)] over a binary relation, this is the set of <i>y</i> with $\langle x, y \rangle \in R$ — exactly what RELATION_JOIN_IMAGE needs.
findTerms (reps)	level reps − 1	all keys at the last component level, <i>but only if</i> reps.back() is a SKOLEM or DUMMY_SKOLEM . In other words the last component acts as a wildcard when it is a Skolem. This is used by applyJoinRule to enumerate the possible witnesses <i>z</i> of JOIN DOWN .

▷ **findTerms** returns the empty vector when the last component is not a Skolem. It is only ever called with a tuple whose last element is the freshly created join witness, so this is intentional — but it is a sharp edge if the function is ever reused elsewhere.

5 The fields of `TheorySetsRels`

Table 5 lists every member. The “life” column is the single most useful thing to know about each: *call* means the field is cleared at the end of **check()** (lines 215–224 of `theory_sets_rels.cpp`); *object* means it is never cleared; *user* means it is a context-dependent container that backtracks with the user context.

Field	Life	Meaning
d_shared_terms <code>CDHashSet<Node></code>	user	Element terms for which a proxy singleton has already been requested from the term registry, so that theory combination will be told about them. Context-dependent because the proxy lemma is asserted in the user context.
d_rel_nodes <code>unordered_set<Node></code>	call	Relational terms whose “upward” member computation has already run in this call. A pure idempotence guard: it makes computeMembersFor... run at most once per term per call, no matter how many memberships trigger it.
d_tuple_reps <code>map<Node, vector<Node>></code>	call	Memoised component-representative vector of a tuple, i.e. the key used to address a TupleTrie . Filled by computeTupleReps .
d_membership_trie <code>map<Node, TupleTrie></code>	call	Per relation <i>representative</i> , the trie of Fig. 2 over its asserted members.
d_symbolic_tuples <code>unordered_set<Node></code>	object	SET_MEMBER nodes whose element argument was a bare variable and which have already been replaced by an explicit tuple constructor (reduceTupleVar). Never cleared, and correctly so: the reduction lemma is a tautology, so emitting it once is enough for the whole solve.
d_rReps_ memberReps_cache <code>map<Node, vector<Node>></code>	call	The membership index. Maps a relation’s representative ρ to the list of <i>tuple representatives</i> currently asserted to be in it.

Field	Life	Meaning
<code>d_rReps_memberReps_exp_cache</code> <code>map<Node,vector<Node>></code>	call	The parallel array of <i>explanations</i> : entry i is the literal (<code>set.member t s</code>) that justifies member i . Note t and s are the terms as they were asserted, <i>not</i> representatives; the gap between s and the relation being reasoned about is closed by adding an equality to every explanation.
<code>d_terms_cache</code> <code>map<Node,map<Kind,vector<Node>>></code>	call	For each set-of-tuples equivalence class, the relational-operator terms in it, bucketed by kind. This is how the solver finds “the relational terms that are equal to the relation this membership is about”.
<code>d_rRep_tcGraph</code> <code>map<Node,map<Node,set<Node>>></code>	call	For a base relation representative ρ : the directed graph G_ρ whose edges are exactly ρ ’s asserted members. <i>Read only by isTCReachable.</i>
<code>d_tcr_tcGraph</code> <code>map<Node,map<Node,set<Node>>></code>	call	For a closure term $T = r^+$: the working graph G_T . Starts as a copy of $G_{\text{rep}(r)}$ and is augmented with edges coming from asserted members of T itself. <i>Read only by doTCInference.</i>
<code>d_tcr_tcGraph_exps</code> <code>map<Node,map<Node,Node>></code>	call	For a closure term T : the explanation of each edge of G_T , keyed by the reconstructed pair term $\langle a, b \rangle$.

5.1 Two maps that are easy to confuse

`d_rRep_tcGraph` is keyed by the *base relation’s representative*; `d_tcr_tcGraph` is keyed by the *closure term itself*. They are initialised to the same graph but they are not the same map and they diverge:

	<code>d_rRep_tcGraph[rep(r)]</code>	<code>d_tcr_tcGraph[r⁺]</code>
edges from members of r	yes	yes
edges from members of r^+	no	yes
read by	isTCReachable	doTCInference
purpose	“is this already derivable?”	“what can I derive?”

Keeping edges of r^+ out of `d_rRep_tcGraph` is essential: that map answers the question “can I skip the expensive TCLOS DOWN splitting lemma for this membership, because it already follows from r ?”. If closure edges leaked into it, the solver would suppress the splitting lemma for a membership it had merely *assumed*, which is circular.

▷ *Because `RelUtils::constructPair(rel,a,b)` builds the pair from the sort of `rel` only, the pair terms produced with `rel_rep`, `tc_rel` and `tc_rel[0]` are the very same node. That is why keys written by `buildTCGraphForRel` (which uses `rel_rep`) can be looked up by `doTCInference` (which uses `tc_rel`). It works, but it relies on an implicit coincidence.*

Part III

The algorithms

6 The top level: check

Algorithm 1 `TheorySetsRels::check(Theory::Effort e)` — lines 57–73

1: **if** $\neg \text{FULLEFFORT}(e)$ **then**

```

2:   return ▷ nothing at lower efforts
3: end if
4: COLLECTRELSINFO ▷ rebuild all indices from the equality engine
5: CHECK ▷ apply the rules, accumulate pending lemmas
6: d_im.doPendingLemmas() ▷ flush

```

The workhorse is the no-argument `check()` (lines 75–246). It has a short preamble that materialises the transitive-closure graphs, then three phases whose boundaries are exactly the down/up split of the calculus.

Algorithm 2 `TheorySetsRels::check()` — lines 75–246

Phase 0 — materialise every closure graph

```

1: for all  $(\rho, K) \in \text{d\_terms\_cache}$  do
2:   for all  $T \in K[\text{TCLOSURE}]$  do
3:     BUILDTCGRAPHFORREL( $T$ ) ▷ the only call site, see Part IV
4:   end for
5: end for

```

Phase A — downward rules, driven by memberships

```

6: for all  $(\rho, \text{members}) \in \text{d\_rReps\_memberReps\_cache}$  do
7:   for all  $i \in [0, |\text{members}|)$  do
8:      $m \leftarrow \text{members}[i]$ ;  $e \leftarrow \text{d\_rReps\_memberReps\_exp\_cache}[\rho][i]$ 
9:      $K \leftarrow \text{d\_terms\_cache}[\rho]$  ▷ relational terms equal to this relation
10:    if  $K$  has TRANSPOSE terms  $tp$  then
11:      APPLYTRANSPORERULE( $tp$ ) ▷ TRANSP EQ: all transposes here are equal
12:      APPLYTRANSPORERULE( $tp[0], \rho, e$ ) ▷ TRANSP DOWN
13:    end if
14:    for all  $t \in K[\text{JOIN}]$  do
15:      APPLYJOINRULE( $t, \rho, e$ )
16:    end for
17:    for all  $t \in K[\text{TABLE\_JOIN}]$  do
18:      APPLYTABLEJOINRULE( $t, \rho, e$ )
19:    end for
20:    for all  $t \in K[\text{PRODUCT}]$  do
21:      APPLYPRODUCTRULE( $t, \rho, e$ )
22:    end for
23:    for all  $t \in K[\text{TCLOSURE}]$  do
24:      APPLYTCRULE( $m, t, \rho, e$ )
25:    end for
26:    for all  $t \in K[\text{JOIN\_IMAGE}]$  do
27:      APPLYJOINIMAGERULE( $m, t, e$ )
28:    end for
29:    for all  $t \in K[\text{IDEN}]$  do
30:      APPLYIDENRULE( $m, t, e$ )
31:    end for
32:  end for
33: end for

```

Phase B — upward rules, driven by terms

```

34: for all  $(\rho, K) \in \text{d\_terms\_cache}$  do

```



```

35:   for all  $(k, terms) \in K$  do
36:     for all  $t \in terms$  do
37:       if  $k \in \{JOIN, PRODUCT, TABLE\_JOIN\}$  then
38:         COMPUTEMEMBERSFORBINOPREL( $t$ )
39:       else if  $k = TRANSPOSE$  then
40:         COMPUTEMEMBERSFORUNARYOPREL( $t$ )
41:       else if  $k = JOIN\_IMAGE$  then
42:         COMPUTEMEMBERSFORJOINIMAGETERM( $t$ )
43:       else if  $k = IDEN$  then
44:         COMPUTEMEMBERSFORIDENTERM( $t$ )
45:       end if
46:     end for
47:   end for
48: end for

```

Phase C — transitive closure forward inference

```

49: DOTCINFERENCE ▷ walk every  $G_T$  and emit TCLOS UP I/II

```

Cleanup

```

50: clear d_tuple_reps, d_rReps_memberReps_exp_cache, d_terms_cache, d_membership_trie,
    d_rel_nodes, d_rReps_memberReps_cache, d_rRep_tcGraph, d_tcr_tcGraph_exps,
    d_tcr_tcGraph

```

▷ Phase B used to be guarded by *if* (`d_rReps_memberReps_cache.find(t_it->first) == ...end()`), i.e. it only ran for equivalence classes with no memberships. That guard was removed in commit `3bcc47a3c` (“Fix upwards closure for relations”, PR #7515, Nov 2021) because upward closure must run even for classes that do have members. The removal was right for join/product/transpose. It also made `buildTCGraphForRel` run a second time for closure terms, which is the subject of Part IV; Phase 0 above is the resolution of that question, and it is why Phase B no longer has a TCLOSURE branch at all.

7 collectRelsInfo: rebuilding the world

Algorithm 3 `collectRelsInfo` — lines 231–314

```

1: for all equivalence classes  $c$  of the equality engine do
2:   for all terms  $n \in c$  do
3:     if  $\text{type}(c) = \text{Bool}$  and  $\text{rep}(c)$  is a constant then ▷  $c$  is the true or false class
4:       if  $n = (\text{set.member } t \ s)$  and  $s$  is a relation then
5:          $\hat{t} \leftarrow \text{rep}(t)$ ;  $\hat{s} \leftarrow \text{rep}(s)$ 
6:         if  $t$  is a variable then
7:           REDUCETUPLEVAR( $n$ ) ▷ lemma:  $n \Leftrightarrow \langle t.1, \dots, t.k \rangle \in s$ 
8:         end if
9:         if  $\text{rep}(c) = \text{true}$  then ▷ negative memberships are ignored
10:          if SAFELYADDTOMAP(d_rReps_memberReps_cache,  $\hat{s}$ ,  $\hat{t}$ ) then
11:            append  $n$  to d_rReps_memberReps_exp_cache[ $\hat{s}$ ]
12:            COMPUTETUPLEREPS( $\hat{t}$ )
13:            d_membership_trie[ $\hat{s}$ ].ADDTERM( $\hat{t}$ , d_tuple_reps[ $\hat{t}$ ])
14:          end if

```

```

15:         end if
16:     end if
17:     else if type(c) is a set of tuples then
18:         if kind(n) is a relational operator then
19:             append n to d_terms_cache[rep(c)][kind(n)]
20:         end if
21:     else if type(c) is a tuple, n neither constant nor variable then
22:         for all components e of n that are not constants do
23:             MAKE_SHARED_TERM(e)
24:         end for
25:     end if
26: end for
27: end for

```

Three details deserve emphasis.

Only positive memberships are indexed. `reason` is computed for both polarities but only the `is_true_eq` branch populates the caches. Negative memberships $\langle a, b \rangle \notin R$ are handled entirely by the equality engine: a rule that derives $\langle a, b \rangle \in R$ will make the literal both true and false, which is the EQ UNSAT rule of the paper.

Everything is keyed by representatives. $\hat{s} = \text{rep}(s)$ and $\hat{t} = \text{rep}(t)$. This is how equality reasoning gets “compiled away”: if $R_1 \approx R_2$, both contribute members to the same bucket, and `d_terms_cache` puts the relational terms of both into the same bucket too. The price is that the explanations must later re-introduce the equalities that were quotiented out — see Section 8.

De-duplication is semantic, not syntactic. `safelyAddToMap` (lines 1664–1691) compares the candidate member against every member already recorded using `areEqual`, not `==`. `areEqual` (lines 1624–1659) is itself recursive on tuples and, as a side effect, *marks non-equal components as shared terms*. So the act of indexing a membership can enqueue proxy lemmas for theory combination. This is deliberate: the join and table-join rules need to be notified when two element terms become equal, and the only way to get that notification is to make them shared.

8 Explanations: the discipline that makes the rest work

```

void TheorySetsRels::sendInfer(Node fact, InferenceId id, Node reason) {
    Node lemma = nodeManager()->mkNode(Kind::IMPLIES, reason, fact);
    d_im.addPendingLemma(lemma, id);
}

```

Every rule builds a *reason* that must be a conjunction of literals that currently hold, and the lemma sent is $\text{reason} \Rightarrow \text{fact}$. Because the indices are keyed by representatives while the explanations record the literals as asserted, essentially every rule ends with the same three-line idiom:

```

Node reason = exp; // exp is (set.member t s), asserted
if (rel != exp[1]) { // s is not syntactically our rel...
    reason = nm->mkNode(AND, reason, // ...so add the equality that says
                        nm->mkNode(EQUAL, rel, exp[1])); // they are the same
}
sendInfer(conclusion, InferenceId::..., reason);

```

Invariant 1 (Explanation soundness). For every call `sendInfer(f, id, r)`, r is a conjunction of literals each of which is entailed by the current assertions (memberships taken from `d_rReps_memberReps_exp_cache`, equalities that hold in the equality engine), and $r \models_{T_R} f$.

There is a safety net for the case where the reason turns out *not* to be entailed — `processInference` (lines 1594–1608) wraps such an inference into the splitting lemma $\neg r \vee f$ instead of asserting f as a fact.

▷ *`processInference` is dead code in the current file: nothing calls it, since `sendInfer` sends everything as a lemma anyway. Likewise `doPendingInfers` is declared in the header but never defined, and the header comments still describe a `d_pending` vector that no longer exists. Reading the header alone will mislead you here.*

9 The non-closure rules

Each operator is implemented by a pair of functions. The table below is the map from the paper to the code; the `InferenceId` column is what you will see in `-stats` output and in `-t sets-rels` traces.

Paper rule	Direction	Function	InferenceId
TRANSP DOWN	down	<code>applyTransposeRule(rel,rep,exp)</code>	<code>SETS_RELS_TRANSPOSE_REV</code>
TRANSP UP	up	<code>computeMembersForUnaryOpRel</code>	<code>SETS_RELS_TRANSPOSE_REV</code>
—	—	<code>applyTransposeRule(vector)</code>	<code>SETS_RELS_TRANSPOSE_EQ</code>
PROD DOWN	down	<code>applyProductRule</code>	<code>SETS_RELS_PRODUCT_SPLIT</code>
PROD UP	up	<code>composeMembersForRels</code>	<code>SETS_RELS_PRODUCE_COMPOSE</code>
JOIN DOWN	down	<code>applyJoinRule</code>	<code>SETS_RELS_JOIN_SPLIT_1/2</code>
JOIN UP	up	<code>composeMembersForRels</code>	<code>SETS_RELS_JOIN_COMPOSE</code>
— (tables)	down	<code>applyTableJoinRule</code>	<code>SETS_RELS_TABLE_JOIN_DOWN</code>
— (tables)	up	<code>applyTableJoinUp</code>	<code>SETS_RELS_TABLE_JOIN_UP</code>
— (iden)	down	<code>applyIdenRule</code>	<code>SETS_RELS_IDENTITY_DOWN</code>
— (iden)	up	<code>computeMembersForIdenTerm</code>	<code>SETS_RELS_IDENTITY_UP</code>
— (join image)	down	<code>applyJoinImageRule</code>	<code>SETS_RELS_JOIN_IMAGE_DOWN</code>
— (join image)	up	<code>computeMembersForJoinImageTerm</code>	<code>SETS_RELS_JOIN_IMAGE_UP</code>
TCLOS DOWN	down	<code>applyTCRule</code>	<code>SETS_RELS_TCLOSURE_UP</code>
TCLOS UP I/II	up	<code>doTCInference</code>	<code>SETS_RELS_TCLOSURE_FWD</code>
— (tuple vars)	—	<code>reduceTupleVar</code>	<code>SETS_RELS_TUPLE_REDUCTION</code>

▷ *The `InferenceId` for `TCLOS DOWN` is called `SETS_RELS_TCLOSURE_UP` and the one for `TCLOS UP I/II` is called `..._FWD`. The naming is inverted with respect to the paper. Do not use the identifiers to orient yourself.*

9.1 Transpose

TRANSP DOWN is a one-liner: reverse the tuple in `exp[0]` and assert it in `tp_rel[0]`. TRANSP UP iterates the members of `rel[0]` and asserts the reverse of each in `rel`.

The extra rule, `applyTransposeRule(vector<Node> tp_terms)`, has no counterpart in the paper. If two transpose terms X^{-1} and Y^{-1} are in the same equivalence class then $X^{-1} \approx Y^{-1}$, and since transpose is injective, $X \approx Y$:

$$\text{TRANSP EQ } \frac{X^{-1} \approx Y^{-1}}{X \approx Y}$$

Note that this is emitted once *per member* of the equivalence class (Algorithm 2 calls it inside the member loop), so with k members and j transposes it enqueues $k(j - 1)$ identical lemmas. They are de-duplicated downstream, but the work is real.

9.2 Product and join, downward

Both split the member tuple. For product, the split is positional: $\langle x_1..x_m, y_1..y_n \rangle \in R_1 \times R_2$ yields $\langle x_1..x_m \rangle \in R_1$ and $\langle y_1..y_n \rangle \in R_2$. For join, a fresh Skolem z is glued into the middle:

Algorithm 4 `applyJoinRule`($J = R_1 \bowtie R_2$, ρ , exp) — lines 1053–1132

```

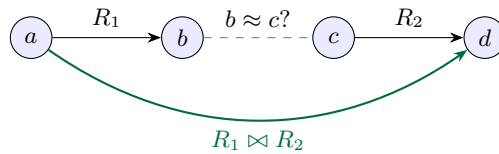
1: if  $J \notin \text{d\_rel\_nodes}$  then
2:   COMPUTEMEMBERSFORBINOPREL( $J$ ); insert  $J$  into  $\text{d\_rel\_nodes}$ 
3: end if
4:  $\langle a_1..a_{m-1}, b_1..b_n \rangle \leftarrow exp[0]$ 
5:  $z \leftarrow \text{MKTYPEDSKOLEM}\text{CACHED}(exp[0], J, \text{SK\_JOIN})$  ▷ same ( $mem, J$ )  $\Rightarrow$  same Skolem
6:  $t_1 \leftarrow \langle a_1..a_{m-1}, z \rangle$ ;  $t_2 \leftarrow \langle z, b_1..b_n \rangle$  ▷ redundancy check: is a witness already present?
7: for all  $u \in \text{d\_membership\_trie}[\text{rep}(R_1)].\text{FINDTERMS}(\text{reps}(t_1))$  do
8:   if  $\text{d\_membership\_trie}[\text{rep}(R_2)].\text{EXISTSTERM}(u \cdot \text{reps}(t_2)_{2..}) \neq \text{null}$  then
9:     return ▷ JOIN DOWN would be a redundant application
10:  end if
11: end for
12: SENDINFER( $t_1 \in R_1$ , JOIN_SPLIT_1, reason)
13: SENDINFER( $t_2 \in R_2$ , JOIN_SPLIT_2, reason)
14: MAKESHAREDTERM( $z$ )

```

The `findTerms/existsTerm` pair implements the paper’s *redundancy criterion* (“an application is redundant if S already contains $\varphi_1[x_1, t], \dots$ for some terms t ”): if some u already witnesses the join for this member, do not introduce a new Skolem. Together with the caching in `SkolemCache` — the witness is keyed by the pair (member, join term), so re-deriving the same membership reuses the same z — this is what stops JOIN DOWN from generating an unbounded supply of witnesses.

9.3 Product and join, upward: `composeMembersForRels`

`computeMembersForBinOpRel` first recurses into the arguments (so that a nested $(X \bowtie Y)^{-1}$ has its members computed bottom-up) and then calls `composeMembersForRels`, which is the quadratic double loop over the members of R_1 and R_2 . For join it must decide whether the “middle” elements match:



- If b and c are currently equal: emit $\langle a, d \rangle \in R_1 \bowtie R_2$, adding $b \approx c$ to the reason when the two terms are syntactically different.
- Otherwise: *do nothing*, but call `makeSharedTerm` on both b and c first. Marking them shared hands the case split to theory combination, which will only split when it actually matters. This replaced an older implementation that emitted an explicit $b \approx c \vee b \not\approx c$ lemma (see PR #7515); the comment in the code spells this out.

9.4 Table join

`RELATION_TABLE_JOIN` carries a `ProjectOp` with an index list $m_1, n_1, \dots, m_k, n_k$ meaning “column m_i of the left tuple must equal column n_i of the right tuple”. `applyTableJoinRule` is the downward rule:

from $e \in (A \bowtie_{\vec{m}, \vec{n}} B)$ it splits e positionally into a and b and concludes $a \in A \wedge b \in B \wedge \bigwedge_i a_{m_i} \approx b_{n_i}$. Unlike JOIN DOWN there is no Skolem, because the join columns are explicit. **applyTableJoinUp** is the matching quadratic upward rule.

9.5 Identity

$\text{iden}(R) = \{\langle x, x \rangle \mid \langle x \rangle \in R\}$ with R unary. Downward (**applyIdenRule**): from $\langle x, y \rangle \in \text{iden}(R)$ conclude $\langle x \rangle \in R \wedge x \approx y$. Upward (**computeMembersForIdenTerm**): from $\langle x \rangle \in R$ conclude $\langle x, x \rangle \in \text{iden}(R)$.

9.6 Join image

$R \text{ jimg } n$ is the set of $\langle x \rangle$ having at least n *distinct* R -successors.

- **Down** (**applyJoinImageRule**): from $\langle x \rangle \in R \text{ jimg } n$ create n fresh Skolems $k_1..k_n$ and conclude $\bigwedge_i \langle x, k_i \rangle \in R \wedge \text{distinct}(k_1..k_n)$. It first short-circuits if the trie already shows $\geq n$ successors of x (**findSuccessors**).
- **Up** (**computeMembersForJoinImageTerm**): scan the members of R , group them by first component, and as soon as n pairwise-distinct second components have been seen for some x , conclude $\langle x \rangle \in R \text{ jimg } n \vee \neg \text{distinct}(y_1..y_n)$. The disjunction is needed because the y_i were only checked to be *not currently known equal*, not to be distinct.

10 Transitive closure in detail

Transitive closure is the only operator whose implementation is not a direct transcription of two rules. It maintains an explicit graph, and the graph is where the interesting invariants live.

10.1 The idea

Read a binary relation r as a directed graph: an asserted membership $\langle a, b \rangle \in r$ is an edge $a \rightarrow b$. Then

$$\langle a, b \rangle \in r^+ \iff \text{there is a path of length } \geq 1 \text{ from } a \text{ to } b.$$

TCLOS UP I says every edge is a path; TCLOS UP II says paths compose. Instead of applying those two rules one step at a time — which would need one full-effort round per path edge — the implementation materialises the graph once and enumerates *all* paths by depth-first search, emitting one SETS_RELS_TCLOSURE_FWD lemma per path. That is **doTCInference**.

Crucially, the graph is not built from the members of r alone. Memberships in r^+ itself are also edges — an assumed $\langle a, b \rangle \in r^+$ can be chained with anything else in r^+ by TCLOS UP II. Those edges are added by **applyTCRule**.

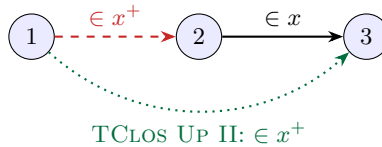


Figure 3: G_{x^+} for the assertions $\langle 2, 3 \rangle \in x$, $\langle 1, 2 \rangle \in x^+$. Solid = edge of the base relation (contributed by **buildTCGraphForRel**), dashed = edge from an asserted closure membership (contributed by **applyTCRule**), dotted = what **doTCInference** derives from the path.

10.2 buildTCGraphForRel: materialising the base graph

Algorithm 5 buildTCGraphForRel(T) where $T = r^+$ — lines 830–877

```

1:  $\rho \leftarrow \text{rep}(r)$ 
2: if  $\rho \notin \text{d\_rReps\_memberReps\_cache}$  then
3:   return ▷  $r$  has no asserted member; nothing to build
4: end if
5:  $\text{members} \leftarrow \text{d\_rReps\_memberReps\_cache}[\rho]$ ;  $\text{exps} \leftarrow \text{d\_rReps\_memberReps\_exp\_cache}[\rho]$ 
6:  $G \leftarrow \emptyset$ ;  $E \leftarrow \emptyset$ 
7: for all  $i \in [0, |\text{members}|)$  do
8:    $u \leftarrow \text{rep}(\text{members}[i].1)$ ;  $v \leftarrow \text{rep}(\text{members}[i].2)$ 
9:   if  $v \notin G[u]$  then
10:     $G[u] \leftarrow G[u] \cup \{v\}$ ;  $E[\langle u, v \rangle] \leftarrow \text{exps}[i]$ 
11:   end if
12: end for
13: if  $|\text{members}| > 0$  then
14:    $\text{d\_rRep\_tcGraph}[\rho] \leftarrow G$  ▷ base graph, for isTCReachable
15:    $\text{d\_tcr\_tcGraph}[T] \leftarrow G$  ▷ overwrites whatever was there
16:    $\text{d\_tcr\_tcGraph\_exps}[T] \leftarrow E$  ▷ overwrites
17: end if

```

Two things to notice. First, the function is a pure function of `d_rReps_memberReps_cache`, which `collectReIsInfo` fills and no rule subsequently modifies. *Calling it at any point during `check()` therefore produces exactly the same graph.* Second, the three assignments at the end are plain `operator[]` assignments: they replace, they do not merge. Remember this for Part IV — it is the whole story of that part.

▷ The early return on line 2 is new. Until 8bcd3239 the function read `d_rReps_memberReps_cache[rel_rep]` and `d_rReps_memberReps_exp_cache[rel_rep]` with `operator[]`, which inserts an empty vector when r has no asserted member. Two maps that Phase A iterates were therefore silently mutated by a graph build, and every later `find( $\rho$ ) != end()` test on them — the standard “does this relation have members?” idiom in this file, used by `computeMembersForJoinImageTerm`, `computeMembersForIdenTerm`, `applyTransposeRule` and `composeMembersForReIs` — reported a false positive for ρ . Those callers happen to survive it, because they go on to iterate a vector that is empty, but the idiom was lying. The same commit dropped the unused local `tc_rel_rep`.

10.3 applyTCRule: TClos Down plus graph augmentation

Algorithm 6 applyTCRule(m, T, ρ, exp) where $T = r^+$ — lines 648–758

```

▷  $G_T$  was built in Phase 0; this rule only ever adds to it
1: if isTCREACHABLE( $m, T$ ) then
2:   return ▷ already entailed; do not split
3: end if
4:  $a \leftarrow \text{rep}(m.1)$ ;  $b \leftarrow \text{rep}(m.2)$ 
5: add edge  $a \rightarrow b$  to  $\text{d\_tcr\_tcGraph}[T]$  and  $\text{exp}$  to  $\text{d\_tcr\_tcGraph\_exps}[T][\langle a, b \rangle]$ 
▷ now the branching rule, with cached Skolems  $s_1, s_2$ 
6:  $\langle x, y \rangle \leftarrow \text{exp}[0]$ 

```

```

7:  $s_1 \leftarrow \text{SK\_TCLOSURE\_DOWN1}(\text{exp}[0], r); \quad s_2 \leftarrow \text{SK\_TCLOSURE\_DOWN2}(\text{exp}[0], r)$ 
8:  $\text{SENDINFER}(\langle x, y \rangle \in r \vee (\langle x, s_1 \rangle \in r \wedge \langle s_2, y \rangle \in r \wedge (s_1 \approx s_2 \vee \langle s_1, s_2 \rangle \in T)), \text{TCLOSURE\_UP},$ 
    $\text{reason})$ 

```

The conclusion is the paper's TCLOS DOWN with the three branches refactored into two: branch 2 ($\langle x, z \rangle \in r, \langle z, y \rangle \in r$) and branch 3 ($\langle x, z_1 \rangle \in r, \langle z_1, z_2 \rangle \in r^+, \langle z_2, y \rangle \in r, z_1 \not\approx z_2$) share the shape $\langle x, s_1 \rangle \in r \wedge \langle s_2, y \rangle \in r$ and are distinguished by $s_1 \approx s_2$ (branch 2, the two-step case) versus $\langle s_1, s_2 \rangle \in r^+$ (branch 3). Note the code does *not* assert $s_1 \not\approx s_2$ in branch 3, so the two branches overlap; that is harmless for soundness and avoids a disequality.

Termination pressure comes from three places, and it is worth being explicit about them because they are the only thing standing between this rule and an infinite regress:

1. **Skolem caching.** s_1, s_2 are produced by `SkolemCache::mkTypedSkolemCached` keyed on $(\text{exp}[0], r)$ and a purpose tag, so re-processing the same membership in a later round reuses the same two Skolems. Without this, every round would invent new nodes.
2. **The reachability guard.** If m is already a member of r , or is connected in $G_{\text{rep}(r)}$, the rule returns before splitting.
3. **Graph augmentation.** Recording $a \rightarrow b$ means the same membership can be discharged by `doTCInference` instead of by splitting on a later round.

▷ *Until 8bcdf3239 this rule opened with a build of its own, under the guard `d_rReps_memberReps_cache.find(tc_rel[0]) != end()` ~~and~~ `tc_rel` $\notin$ `d_rel_nodes` ~~and~~ `getRepresentative(tc_rel[0])` $\notin$ `d_rRep_tcGraph`. Note the first clause looked up `tc_rel[0]`, the term itself, where `buildTCGraphForRel` looks up its representative: when r was not its own representative the guard was simply false, no graph was built here, and `isTCReachable` below ran against nothing — so the rule emitted a splitting lemma for a membership the graph would have discharged. Phase 0 sidesteps the whole question by building unconditionally, before any rule can look.*

10.4 isTCReachable: the entailment guard

Algorithm 7 `isTCReachable( $m, T$ )` — lines 747–776

```

1: if  $m$  occurs in d_rReps_memberReps_cache[rep( $r$ )] then
2:   return true ▷  $m \in r$ , so  $m \in r^+$  by TCLOS UP I
3: end if
4: if rep( $r$ )  $\in$  d_rRep_tcGraph then
5:   return DFS from rep( $m.1$ ) to rep( $m.2$ ) in d_rRep_tcGraph[rep( $r$ )]
6: end if
7: return false

```

The DFS is the second `isTCReachable` overload (lines 778–814): standard depth-first search with a `hasSeen` set, writing its answer into a by-reference `bool`. Two small inefficiencies: the loop does not break once `isReachable` becomes true in a recursive call, and the caller applies `getRepresentative` twice to the same node.

Note the graph consulted is `d_rRep_tcGraph`, the *base* graph. This is the asymmetry described in Part II: it must not see edges that came from assumed r^+ memberships, otherwise a membership would justify suppressing its own splitting lemma.

10.5 doTCInference: enumerating paths

Phase C consists of three overloads that form a driver / per-graph / recursive triple.

Algorithm 8 doTCInference (three overloads) — lines 1255–1273, 860–897, 899–978

```

1: function DOTCINFERENC ▷ driver, line 1255
2:   for all  $(T, G) \in \text{d\_tcr\_tcGraph}$  do
3:     DOTCINFERENC( $G, \text{d\_tcr\_tcGraph\_exps}[T], T$ )
4:   end for
5: end function

6: function DOTCINFERENC( $G, E, T$ ) ▷ per graph, line 860
7:   for all  $u \in \text{dom}(G)$  do
8:     for all  $v \in G[u]$  do
9:       DFS( $T, [E[\langle u, v \rangle]], G, E, u, v, \{u\}$ )
10:    end for
11:  end for
12: end function

13: function DFS( $T, \text{reasons}, G, E, \text{start}, \text{cur}, \text{seen}$ ) ▷ line 899
14:   $x \leftarrow \text{reasons.front}()[0].1; \quad y \leftarrow \text{reasons.back}()[0].2$ 
15:   $\text{all} \leftarrow \text{reasons}$ 
16:  for all consecutive pairs  $(\ell_i, \ell_{i+1})$  in  $\text{reasons}$  do
17:    if  $\ell_i[0].2 \neq \ell_{i+1}[0].1$  then
18:      append  $\ell_i[0].2 \approx \ell_{i+1}[0].1$  to  $\text{all}$  ▷ stitch the chain
19:    end if
20:    if  $T \neq \ell_i[1]$  and  $r \neq \ell_i[1]$  then
21:      append  $r \approx \ell_i[1]$  to  $\text{all}$  ▷ the literal was about an equal relation
22:    end if
23:  end for
24:  (same relation-equality fix-up for  $\text{reasons.back}()$ )
25:  SENDINFER( $\langle x, y \rangle \in T, \text{TCLOSURE\_FWD}, \bigwedge \text{all}$ )

26:  if  $\text{cur} \in \text{seen}$  then
27:    return
28:  end if
29:   $\text{seen} \leftarrow \text{seen} \cup \{\text{cur}\}$ 
30:  for all  $w \in G[\text{cur}]$  do
31:    DFS( $T, \text{reasons} \cdot E[\langle \text{cur}, w \rangle], G, E, \text{start}, w, \text{seen}$ )
32:  end for
33: end function

```

reasons is the chain of membership literals along the current path; the conclusion is always “first element of the first literal” to “second element of the last literal”. Two subtleties:

- The endpoints x, y are read from the *literals*, not from the graph nodes. Graph nodes are representatives, but the literals mention the original terms; taking the endpoints from the literals is what makes the emitted membership talk about terms the rest of the solver has seen. The equalities appended in the loop close the gap between consecutive links.
- *seen* is passed *by reference* and is shared across siblings in the DFS. It therefore behaves as a global visited-set for the whole traversal rooted at one edge, not as a path-local set. This makes

the traversal linear rather than exponential, at the cost of not enumerating every simple path — a deliberate under-approximation. Since every emitted lemma is sound, missing some paths only costs completeness of the round, and the missing memberships are typically re-derived on later rounds from the newly asserted ones.

Example 1 (The graph of Figure 3). $G_{x^+} = \{1 \mapsto \{2\}, 2 \mapsto \{3\}\}$ with $E[\langle 1, 2 \rangle] = (\langle 1, 2 \rangle \in x^+)$, $E[\langle 2, 3 \rangle] = (\langle 2, 3 \rangle \in x)$. The driver visits $u = 1$: **dfs** with reasons $[\langle 1, 2 \rangle \in x^+]$ emits $\langle 1, 2 \rangle \in x^+$ (trivial), then recurses to $cur = 2$ with reasons $[\langle 1, 2 \rangle \in x^+, \langle 2, 3 \rangle \in x]$ and emits

$$\langle 1, 2 \rangle \in x^+ \wedge \langle 2, 3 \rangle \in x \Rightarrow \langle 1, 3 \rangle \in x^+,$$

which together with an assertion $\langle 1, 3 \rangle \notin x^+$ closes the branch.

10.6 The transitive-closure invariants

Invariant 2 (Node domain). Every vertex of $\mathbf{d_rRep_tcGraph}[\rho]$ and of $\mathbf{d_tcr_tcGraph}[T]$ is an equivalence-class representative of an element term. Consequently two currently-equal elements are the *same* vertex, and graph reachability already accounts for equality reasoning.

Invariant 3 (Base graph). $\mathbf{d_rRep_tcGraph}[\rho] = \{\text{rep}(a) \rightarrow \text{rep}(b) \mid \langle a, b \rangle \text{ is an asserted member of the class } \rho\}$ — edges of the *base relation only*, never of a closure term.

Invariant 4 (Working graph soundness). For $T = r^+$, every edge $a \rightarrow b$ of $\mathbf{d_tcr_tcGraph}[T]$ satisfies $\langle a, b \rangle \in T$ under the current assertions: either $\langle a, b \rangle$ is an asserted member of r 's class (TCLOS UP I) or an asserted member of T 's class. Hence *every path in G_T from a to b witnesses $\langle a, b \rangle \in T$* by TCLOS UP II, and the chain of edge explanations is a valid reason for it.

Invariant 5 (Working graph completeness). $\mathbf{d_tcr_tcGraph}[T] \supseteq \mathbf{d_rRep_tcGraph}[\text{rep}(r)]$, and contains an edge for every asserted member of T that **isTCReachable** did not already discharge.

Invariant 6 (Explanation totality). $\text{dom}(\mathbf{d_tcr_tcGraph_exps}[T]) \supseteq$ the edge set of $\mathbf{d_tcr_tcGraph}[T]$, with keys built by `RelsUtils::constructPair`. **doTCInference** dereferences `find(new_pair)->second` without a null check, so a violation of this invariant is undefined behaviour, not a wrong answer.

Invariant 7 (Lifetime). All three TC maps are scratch for a single **check()** call and are cleared at lines 243–245. Nothing in them survives to the next full-effort round.

Invariants 2, 3, 4, 6 and 7 hold in any arrangement of the code. Invariant 5 is the delicate one: it holds with Phase 0 by construction, because the only assignment to $\mathbf{d_tcr_tcGraph}[T]$ happens before any rule runs and everything afterwards is an insertion, whereas on **main** it takes a guard at each of two call sites and holds only per base relation. That is the subject of the next part.

Part IV

Should `buildTCGraphForRel` be called once or twice?

11 The question

On **main** (5cc03f4b9, whose line numbers this section quotes), **buildTCGraphForRel** has two call sites, each guarded:

	Site	Guard
★ ₁	<code>applyTCRule</code> , line 645	<code>d_rReps_memberReps_cache</code> contains <code>tc_rel[0]</code> and <code>tc_rel</code> $\notin$ <code>d_rel_nodes</code> and <code>rep(tc_rel[0])</code> $\notin$ <code>d_rRep_tcGraph</code>
★ ₂	<code>check()</code> Phase B, line 191	<code>*term_it</code> $\notin$ <code>d_rel_nodes</code> and <code>rep(*term_it)[0]</code> $\notin$ <code>d_rRep_tcGraph</code>

in code:

```
// Protect d_tcr_tcGraph from being overwritten, if it already exists
if (d_rel_nodes.find(*term_it) == d_rel_nodes.end()
    && d_rRep_tcGraph.find(getRepresentative((*term_it)[0]))
    == d_rRep_tcGraph.end())
{
    buildTCGraphForRel(*term_it);
    d_rel_nodes.insert(*term_it);
}
```

The two maps in those conditions are not being consulted for their contents; they are being used as “a graph for this relation has already been built” markers, and the comment says what they are protecting against. So the guards are not an optimisation, they are the mechanism that keeps the second call from destroying the first call’s work (Observation 2 below). The question of this part is whether to keep that arrangement or to replace both sites with a single unconditional pass immediately after `collectRelsInfo`.

`main` is correct on everything measured here (Section 15), so what follows is not a bug report. Two properties distinguish the two configurations:

- **Which call builds is decided by iteration order.** The outcome depends on whether Phase A reached T before Phase B did. Correctness is preserved by the facts that Phase A always runs first and that $\star_1$ builds before it augments — a two-site argument, where one call needs no argument at all.
- **The guards are keyed by base representative, not by term.** If two closure terms x^+ and y^+ have $x \approx y$, the second of them never receives the base edges in `d_tcr_tcGraph`, only whatever $\star_1$ later inserts. Invariant 5 fails for that term. This is benign — the two closure terms are congruent, so the memberships derived for the first propagate — and it is unobservable in the test suite, but it is a weaker statement than the invariant.

12 Static analysis

Observation 1 (The graph does not depend on when it is built). `d_rReps_memberReps_cache` and `d_rReps_memberReps_exp_cache` are written only by `collectRelsInfo`. No rule in Phase A or Phase B adds a member to them (rules only enqueue *lemmas*, which become visible in the next full-effort round). `buildTCGraphForRel` reads nothing else. Therefore $\star_1$ and $\star_2$ compute **bit-identical** graphs, and a single call placed anywhere before Phase C computes the same graph too.

Observation 2 (A second call would be destructive). Between $\star_1$ and $\star_2$, `applyTCRule` adds edges to `d_tcr_tcGraph[T]` for the asserted members of T itself (Algorithm 6, line 8). Were $\star_2$ to run, it would execute `d_tcr_tcGraph[tc_rel] = rel_tc_graph`, a whole-map assignment, and **all of those edges would be discarded**, together with their explanations. This is what the guards on `main` exist to prevent, and it is why they cannot be dropped while the call site remains.

So the second call is not a redundant recomputation of the same thing. It is a recomputation of a *smaller* thing that would overwrite a larger one. The only reason the resulting bug was survivable for as long as it was is if `(members.size() > 0)` inside `buildTCGraphForRel`: the damage needs the base relation to have at least one asserted member, and most transitive-closure regressions assert memberships in r^+ or in r but rarely both.

Observation 3 (Cost). When $\star_2$ does fire — for every closure term the guards have not already marked — it is $O(|members(r)|)$ node constructions and map insertions per closure term per full-effort round, including one `RelsUtils::constructPair APPLY_CONSTRUCTOR` node per member, which goes through the node manager’s hash-consing table. Phase 0 does that work exactly once

after $\star_1 + \text{applyTCRule}$



after a rebuild of G_{x^+}



build once, up front

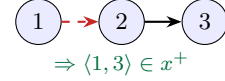


Figure 4: What Observation 2 would cost on the benchmark of Section 13: rebuilding G_{x^+} after **applyTCRule** has run loses the edge $1 \rightarrow 2$, and with it the only path from 1 to 3.

instead, and the guard lookups on the hot path of **applyTCRule** — three map searches and a **getRepresentative** per (membership, closure term) pair — disappear with it. It is not dominant, and Section 15 finds it unmeasurable.

13 The benchmarks that exercise the interaction

```
; EXPECT: unsat
(set-logic ALL)
(declare-fun x () (Relation Int Int))
; base edge: (2,3) in x, hence (2,3) in (rel.tclosure x)
(assert (set.member (tuple 2 3) x))
; closure edge not derivable from x alone
(assert (set.member (tuple 1 2) (rel.tclosure x)))
; transitivity of the two forces (1,3) in (rel.tclosure x)
(assert (not (set.member (tuple 1 3) (rel.tclosure x))))
(check-sat)
```

This is exactly TCLOS UP I followed by TCLOS UP II, i.e. Example 3 of the paper with one of the two premises moved from R to R^+ . The problem is trivially unsatisfiable, and it is the smallest formula in which G_{x^+} has to hold a base edge and a closure edge at the same time: lose the closure edge $1 \rightarrow 2$ and **doTCInference** can only re-derive $\langle 2, 3 \rangle \in x^+$, leaving the solver to split the TCLOS DOWN disjunction of **applyTCRule** over freshly reachable Skolems for ever. `regress1/rels/tc-overwrite-bug.smt2` is the same formula, added to the suite by PR #12793.

Four variations pin the shape down. All were run on the same Debug build with a 20s limit; both configurations answer all five correctly, which is the point — `main`'s guards and Phase 0 protect the same edges.

Benchmark	Shape	result / time	
		main	once
A repro	$\langle 2, 3 \rangle \in x$, $\langle 1, 2 \rangle \in x^+$, $\langle 1, 3 \rangle \notin x^+$	unsat 0.013 s	unsat 0.012 s
B no-base-member	as A but $\langle 2, 3 \rangle \in x^+$ instead of $\in x$	unsat 0.011 s	unsat 0.011 s
C all-base	$\langle 1, 2 \rangle, \langle 2, 3 \rangle \in x$, $\langle 1, 3 \rangle \notin x^+$	unsat 0.010 s	unsat 0.009 s
D sat-no-path	$\langle 3, 4 \rangle \in x$, $\langle 1, 2 \rangle \in x^+$, $\langle 1, 4 \rangle \notin x^+$	sat 0.011 s	sat 0.010 s
E chain4	$1 \rightarrow 2(x^+)$, $2 \rightarrow 3(x)$, $3 \rightarrow 4(x^+)$, $4 \rightarrow 5(x)$, $\langle 1, 5 \rangle \notin x^+$	unsat 0.010 s	unsat 0.010 s

- **B** isolates the `members.size() > 0` guard inside **buildTCGraphForRel**. Moving the one base membership into the closure makes r memberless, so a rebuild would store nothing and could not overwrite anything; the closure edges survive with or without protection.
- **C** is Example 3 of [1] verbatim: all edges are base edges, so no closure-only edge is ever at risk.
- **D** checks that neither configuration *over*-infers: there is no path from 1 to 4, the problem is satisfiable, and both say **sat**.
- **E** is the same interaction over a four-edge path, i.e. with two closure edges to preserve rather than one.

13.1 The in-tree regression that used to be disabled

test/regress/cli/regress0/rels/rel_tc_7.cvc.smt2 is the same interaction reached through a join:

```
; EXPECT: unsat
(set-logic ALL)
(declare-fun x () (Relation Int Int))
(declare-fun y () (Relation Int Int))
(assert (= y (rel.join (rel.tclosure x) x)))
(assert (set.member (tuple 1 2) (rel.join (rel.join x x) x)))
(assert (not (set.subset y (rel.tclosure x))))
(check-sat)
```

until recently test/regress/cli/CMakeLists.txt listed it in regression_disabled_tests:

```
# timeout after fixing upwards closure for relations
regress0/rels/rel_tc_7.cvc.smt2
```

git show 3bcc47a3c - test/regress/CMakeLists.txt shows that this very line was added by PR #7515, whose description says as much: “*It disables a regression that times out after these fixes.*” The join and the outer closure put memberships into both x ’s class and x^+ ’s class, which is benchmark A again, so protecting the closure edges is exactly what the test needs.

Configuration	rel_tc_7.cvc.smt2	Time
main (5cc03f4b9)	unsat	0.01 s
once (8bcd3239)	unsat	0.01 s

It has been passing since PR #12793 landed, but nobody noticed, so it stayed disabled. Commit 8bcd3239 moves it out of regression_disabled_tests into regress_0_tests, where it passes in 0.13s under `ctest` and also under `-check-proofs`, `-check-unsat-cores` and `-produce-proofs-proof-granularity=theory-rewrite`, which is what the other CI configurations run. Nothing else in the disabled list changes: regress1/rels/garbage_collect.cvc.smt2 still times out in both configurations.

14 The patch

1. Add an unconditional pass at the top of `check()` — i.e. immediately after `collectRelsInfo` in the caller — that builds the graph of every closure term once (Phase 0 of Algorithm 2).
2. Delete ★₁ from `applyTCRule` (and the now-pointless `d_rel_nodes.insert(tc_rel)`).
3. Delete ★₂ from Phase B — the whole branch, guard included, so that Phase B no longer has a TCLOSURE case at all.
4. While there: make `buildTCGraphForRel` look the membership list up with `find` instead of `operator[]`, so that it stops inserting empty entries into the two maps Phase A iterates, and drop the unused `tc_rel_rep`.

```
// --- in TheorySetsRels::check(), before Phase A -----
// Build the transitive closure graph of every RELATION_TCLOSURE term before
// any rule is applied.
//
// buildTCGraphForRel reads only d_rReps_memberReps_cache,
// d_rReps_memberReps_exp_cache and the equality engine. The two caches are
// filled by collectRelsInfo above and are not written by any rule, and the
// rules only queue pending lemmas (see sendInfer), so representatives do not
// change either. The graphs built here are therefore exactly the graphs the
// rules below would have built lazily.
//
```

```

// Building them here, once per tc term, is what makes the graphs stable:
// applyTCRule adds the memberships of TC(r) itself to d_tcr_tcGraph, and any
// later call to buildTCGraphForRel would overwrite that graph with the base
// members of r alone, so those edges would never reach doTCInference and the
// TCLOSURE-UP II (transitivity) inferences would be lost.
for (const std::pair<const Node, std::map<Kind, std::vector<Node> > >& t :
    d_terms_cache)
{
    std::map<Kind, std::vector<Node> >::const_iterator tc_it =
        t.second.find(Kind::RELATION_TCLOSURE);
    if (tc_it != t.second.end())
    {
        for (const Node& tc_rel : tc_it->second) { buildTCGraphForRel(tc_rel); }
    }
}

// --- in buildTCGraphForRel -----
MEM_IT mem_it = d_rReps_memberReps_cache.find(rel_rep);
if (mem_it == d_rReps_memberReps_cache.end())
{
    // tc_rel[0] has no asserted members, nothing to build
    return;
}
const std::vector<Node>& members = mem_it->second;

```

The header documentation of `d_rRep_tcGraph` and `buildTCGraphForRel` is updated in the same commit: PR #12793 had documented `d_rRep_tcGraph` as doubling as a “graph already built” marker that callers must test, which is no longer true of either map.

Ordering matters: the pass must run *before* Phase A, because `applyTCRule` consults `d_rRep_tcGraph` through `isTCReachable` to decide whether to skip the expensive splitting lemma, and it appends to `d_tcr_tcGraph`.

Deleting the Phase B branch *entirely* matters, rather than keeping it and dropping its now-apparently-redundant guard. Phase 0 makes the *build* redundant; it never makes the overwrite harmless, because `applyTCRule` has run in between (Observation 2). A call site protected only by bookkeeping state is an invitation to exactly that mistake, which is the strongest argument for not having one.

14.1 Why not keep the guards?

main’s arrangement — keep both lazy call sites, protect the graph with the `d_rel_nodes` and `d_rRep_tcGraph` markers — is the smaller change, and it works. Three things make the up-front pass the better resting place:

- It removes the ★₁ guard’s representative bug. That guard tests `d_rReps_memberReps_cache.find(tc_rel[0])` while `buildTCGraphForRel` looks up `rep(tc_rel[0])`, so when *r* is not its own representative ★₁ does not fire: `isTCReachable` sees no graph during Phase A and `applyTCRule` emits a TCLOS DOWN splitting lemma for a membership the graph would have discharged. Phase 0 always has the graph in place before the first membership is processed.
- It replaces two call sites and a four-clause condition, whose correctness depends on Phase A preceding Phase B, with “built once, before any rule, then only added to” — Invariant 5, for every closure term rather than one per base representative.
- It saves the redundant build and the per-membership guard lookups (Observation 2). Phase B visited every closure term anyway, so nothing new is computed.

15 Measurement

15.1 Setup

What has to be shown is the opposite of a win: that replacing the guards with Phase 0 changes *nothing* observable. Both binaries were built in the same tree, 5cc03f4b9 for `main` and 8bcd3239 for `once` (`CMAKE_BUILD_TYPE=Debug`, `ENABLE_TRACING=ON`, Apple clang on macOS/arm64). Absolute times are therefore much larger than a production build would give; only the comparison between the two columns is meaningful.

▷ *A methodology trap worth recording, because it produced a completely wrong first set of numbers. `cvc5` builds `bin/cvc5` against `@rpath/libcvc5.1.dylib` with an `LC_RPATH` pointing at the absolute build directory. Copying only `build/bin/cvc5` aside and then rebuilding makes the copied “old” executable load the new library, so both binaries run the same code and every benchmark looks identical. Each configuration must be snapshotted together with `libcvc5.1.dylib` and `libcvc5parser.1.dylib`, with `install_name_tool` rewriting the `rpath` to the snapshot directory (and an ad-hoc `codesign` afterwards on macOS).*

15.2 Answers and inference counts

The sharpest available check is not pass/fail but the per-inference counters: two configurations that emit the same number of lemmas of every `InferenceId`, on every benchmark, are doing the same reasoning. The comparison covers all 100 in-tree `regress0/rels` and `regress1/rels` benchmarks plus the 67 generated closure benchmarks of `benchmarks/gen_stress.py` (below), 20 s limit each, comparing the answer line and the whole `theory::sets::inferences/inferencesLemma` histogram.

	main vs once
benchmarks compared	167
of which return a definitive answer	165
answers differing	0
<code>theory::sets</code> inference histograms differing	0
<code>ctest -R rels</code> (with <code>rel_tc_7</code> re-enabled)	99/99 pass
<code>ctest -R "sets rels relation"</code>	303/303 pass

The two non-definitive benchmarks are the same in both configurations: the expected parse error of `issue11903-join-unit-tuple.smt2` and the timeout of the disabled `garbage_collect.cvc.smt2`.

15.3 Scaling families

The in-tree closure benchmarks are all tiny, so seven families were generated at $N \in \{4, 8, 16, 32, 64\}$ to put load on the code that Phase 0 touches: `base-chain` (a chain of N base edges, query at the far end), `closure-chain` (the same chain asserted in x^+ instead), `mixed-chain` (alternating, i.e. the shape that needs the edges `applyTCRule` contributes), `clique` (edges $i \rightarrow i+1$ and $i \rightarrow i+2$), `eqrel` (three provably equal base relations), `sat-chain` (a satisfiable query) and `join-tc` (the `rel_tc_7` shape at size N). Times are the wall clock of the largest instance of each family; the last row is all 67 generated benchmarks, including the equal-relation probes of the next subsection.

Family, $N = 64$	main	once	ratio
base-chain	0.210 s	0.208 s	1.01
closure-chain	0.231 s	0.234 s	0.99
mixed-chain	0.223 s	0.221 s	1.01
clique	0.564 s	0.559 s	1.01
eqrel	0.213 s	0.211 s	1.01
sat-chain	1.210 s	1.208 s	1.00
join-tc	2.283 s	2.241 s	1.02
all 67 generated	8.663 s	8.594 s	1.01

There is no measurable difference, and none should be expected: the work Phase 0 removes is a handful of map lookups per membership (the $\star_1$ guard) and one wasted rebuild per closure term whose base relation has members, against a solve time dominated by the $O(N^2)$ TCLOS UP lemmas that **doTCInference** emits either way.

15.4 The one case where once could do more work

Phase 0 builds one graph per closure *term*; **main**’s guards admit at most one build per base *representative*. Where several closure terms sit over provably equal base relations, Phase 0 therefore populates **d_tcr_tcGraph** for all of them and **doTCInference** walks each — k times the forward inferences instead of one. Two probes bound the risk:

- Three families were generated with k closure terms over base relations asserted equal: **eqwide** (plain top-level equalities, $k \in \{2, 4, 8, 16\}$, $N \in \{16, 32, 64\}$), **eqterm** (each closure term pinned by an auxiliary relation $t_k = x_k^+$, same grid) and **eqsat** (equalities entailed only at the SAT level, so the preprocessor cannot substitute them, $k \in \{1, 2, 4, 8\}$, $N \in \{16, 32\}$). The 24 instances of the first two total 2.311 s and 2.300 s, a ratio of 1.01 like everything else and flat in k ; on **eqsat** the SETS_RELS_TCLOSURE_FWD count is 528 at $N = 32$ in both, and *independent of* k , which is the tell. Tracing with **-trace=rels-debug** confirms it: preprocessing collapses the equal relations and **check()** still reports “Check 1 terms of kind **rel.tclosure**”.
- Across all 100 in-tree **rels** benchmarks, no equivalence class ever contains more than one **rel.tclosure** term.

So the case is real in principle and did not occur once in anything measurable. Should it ever matter, the fix is to key Phase 0 by base representative, which recovers **main**’s work profile without reintroducing a second call site.

15.5 Differential fuzzing against an exact oracle

The in-tree suite barely exercises the combination of a base edge and a closure edge in the same graph, so a targeted random test was added. Formulas have the shape

$$\{\langle u_i, v_i \rangle \in x\}_{i \in I} \cup \{\langle u_j, v_j \rangle \in x^+\}_{j \in J} \cup \{\langle a, b \rangle \notin x^+\}$$

over pairwise-distinct integer constants, with 2–7 nodes and 1–10 edges, biased so that both labels usually occur.

The oracle is exact. Let E be the union of both edge sets. In any model, each asserted x -edge is in x and each asserted x^+ -edge $\langle u, v \rangle$ forces some x -path $u \rightsquigarrow v$; hence b reachable from a in E (in ≥ 1 steps) implies $\langle a, b \rangle \in x^+$, so the formula is unsatisfiable. Conversely, taking x to be the asserted x -edges plus one direct edge per asserted x^+ -edge gives $x^+ =$ the transitive closure of E , which is a model whenever b is *not* reachable from a in E . So

$$\text{formula unsatisfiable} \iff b \text{ reachable from } a \text{ in } E \text{ in } \geq 1 \text{ steps,}$$

which a five-line depth-first search decides. *Any* disagreement with it is a bug in one of the two configurations, so this test does not merely compare them against each other.

Result 900 instances over seeds 1–3, 8 s limit each.

	main	once
matches the oracle	900	900
timeout	0	0
<i>wrong answer</i>	0	0
answers differing between the two	0	

Both configurations decide every instance the oracle decides, and neither ever produced a wrong answer — in particular, having the closure edges present never turned a satisfiable instance into unsat, which is the failure mode to worry about when a rule gets more to work with.

16 Conclusion of the investigation

Building once, immediately after collectReIsInfo, is better, but not for the reason one might expect: it is not a speedup and not a bug fix. Against **main** the measurements say plainly that nothing observable changes — same answers, same inference histograms, same run times to within noise (Section 15). The case is structural:

1. **The invariant becomes checkable by inspection.** Invariant 5 — “ G_T is built exactly once, before any rule runs, and is only ever added to afterwards” — holds by construction, for every closure term. **main** establishes something weaker (per base representative, and only because Phase A happens to precede Phase B), and it spends two call sites and a four-clause condition doing it.
2. **It removes the coupling of correctness to bookkeeping state.** `d_rel_nodes` and `d_rRep_tcGraph` stop doubling as “already built” markers. That coupling is the kind that fails quietly: the protection is invisible at the call site, so the next person to work on Phase B can reasonably conclude the condition is a redundant optimisation and drop it, and no test they are likely to run will complain until a formula asserts memberships in both r and r^+ .
3. **It removes a latent representative bug** in the $\star_1$ guard (Section 14.1), and stops `buildTCGraphForRel` from mutating the membership maps through `operator[]`.
4. **It revives an in-tree regression.** `rel_tc_7.cvc.smt2` has been passing since PR #12793 and is re-enabled here (Section 13.1), which puts the interaction under CI in every configuration rather than in a benchmark directory.

The only argument for keeping a lazy build is to avoid constructing graphs for closure terms that no rule ever consults. That argument does not apply here: Phase C (**doTCInference**) iterates *all* of `d_tcr_tcGraph` unconditionally, so every graph is consulted in every round anyway — and Phase B built all of them already.

17 Other rough edges noticed along the way

None of these are addressed by the patch; they are recorded because a newcomer will trip over them.

1. **doPendingInfers** is declared in the header and never defined; **processInference** is defined and never called; the header comments refer to a `d_pending` vector that does not exist.

2. The header declares `applyJoinImageRule(Node mem_rep, Node rel_rep, Node exp)` and `applyIdenRule(Node mem_rep, Node rel_rep, Node exp)`, but both are called with the *operator term*, not a representative, and the definitions name the parameter accordingly.
3. `start_node_rep` in the recursive `doTCInference` and the `*_rel_rep` parameters of `applyTransposeRule`, `applyProductRule`, `applyJoinRule` and `applyTCRule` are unused except in trace output. (`tc_rel_rep` in `buildTCGraphForRel` was the same and is gone as of 8bcd3239.)
4. `doTCInference` dereferences `rel_tc_graph_exps.find(new_pair)->second` without checking for `end()` (Invariant 6).
5. `applyTransposeRule(vector)` is invoked once per member of the equivalence class although its conclusion does not depend on the member.
6. The DFS in `isTCReachable` does not stop early once the target is found.

A Quick reference: file map

All line numbers in this document refer to `src/theory/sets/theory_sets_rels.cpp` at commit 8bcd3239 (branch `build-trc-graph-once`), except in Part IV where a passage quotes `main` (5cc03f4b9).

Line	Function	Role
57	<code>check(Effort)</code>	entry point; full effort only
75	<code>check()</code>	Phase 0 / A / B / C driver, Alg. 2
252	<code>collectRelsInfo</code>	rebuild indices from the equality engine
343	<code>computeMembersForJoinImageTerm</code>	join image, up
468	<code>applyJoinImageRule</code>	join image, down
542	<code>applyIdenRule</code>	identity, down
584	<code>computeMembersForIdenTerm</code>	identity, up
648	<code>applyTCRule</code>	TCLOS DOWN + graph augmentation
761	<code>isTCReachable</code>	entailment guard (2 overloads)
830	<code>buildTCGraphForRel</code>	materialise $G_{\text{rep}(r)}$ and G_T
879	<code>doTCInference(G, E, T)</code>	per-graph path enumeration
918	<code>doTCInference(recursive)</code>	DFS, builds the explanation chain
1008	<code>applyProductRule</code>	PROD DOWN
1072	<code>applyJoinRule</code>	JOIN DOWN + redundancy check
1153	<code>applyTableJoinRule</code>	table join, down
1222	<code>applyTransposeRule(vector)</code>	TRANSP EQ
1243	<code>applyTransposeRule(rel,rep,exp)</code>	TRANSP DOWN
1274	<code>doTCInference()</code>	Phase C driver
1295	<code>computeMembersForBinOpRel</code>	recurse into args, then compose
1355	<code>computeMembersForUnaryOpRel</code>	TRANSP UP
1413	<code>composeMembersForRels</code>	JOIN UP / PROD UP
1526	<code>applyTableJoinUp</code>	table join, up
1613	<code>processInference</code>	(dead) entailment-checked assert
1643	<code>areEqual</code>	recursive on tuples; marks shared terms
1683	<code>safelyAddToMap</code>	semantic de-duplication of members
1712	<code>makeSharedTerm</code>	request a proxy singleton
1730	<code>computeTupleReps</code>	memoise a tuple's representative vector
1746	<code>reduceTupleVar</code>	replace a tuple variable by a constructor
1774	<code>TupleTrie::*</code>	the membership index
1901	<code>sendInfer</code>	enqueue <i>reason</i> $\Rightarrow$ <i>fact</i>

B Reproducing the experiment

```
# Snapshot a configuration together with its shared libraries, otherwise the
# copied executable will load whatever libcvc5.dylib is in build/src later on.
# Keeping the build tree's own bin/ src/ src/parser/ layout means a single
# -rpath rewrite per entry is enough.
snapshot() { # $1 = destination directory
  R=$PWD; d=$1; rm -rf $d; mkdir -p $d/bin $d/src/parser
  cp $R/build/bin/cvc5 $d/bin/
  cp -R $R/build/src/libcvc5*.dylib $d/src/
  cp -R $R/build/src/parser/libcvc5parser*.dylib $d/src/parser/
  install_name_tool -rpath $R/build/src $d/src \
    -rpath $R/build/src/parser $d/src/parser $d/bin/cvc5
  codesign -f -s - $d/bin/cvc5 # required on arm64
}

git stash # baseline = main
ninja -C build cvc5-bin && snapshot /tmp/base
git stash pop # once
ninja -C build cvc5-bin && snapshot /tmp/once

# Both configurations answer everything, so compare the answers *and* the
# per-InferenceId histograms, which is the sensitive check.
python3 rels-guide/benchmarks/gen_stress.py /tmp/stress # 67 benchmarks
for f in test/regress/cli/regress[01]/rels/*.smt2 /tmp/stress/*.smt2; do
  a=$(/tmp/base/cvc5 --tlimit=20000 --stats $f 2>&1 \
    | grep -E '^(sat|unsat|unknown)|theory::sets::inferences' | sort)
  b=$(/tmp/once/cvc5 --tlimit=20000 --stats $f 2>&1 \
    | grep -E '^(sat|unsat|unknown)|theory::sets::inferences' | sort)
  [ "$a" = "$b" ] || echo "DIFF $f"
done

BASE_BIN=/tmp/base/bin/cvc5 ONCE_BIN=/tmp/once/bin/cvc5 \
MAX_NODES=7 MAX_EDGES=10 \
python3 rels-guide/benchmarks/fuzz_tc.py 1 300 # seeds 1, 2, 3
```

References

- [1] B. Meng, A. Reynolds, C. Tinelli, C. Barrett. *Relational Constraint Solving in SMT*. CADE-26, LNCS 10395, Springer, 2017. (MRT+17.pdf; the calculus of Section 2 is Figures 2 and 3 of that paper.)
- [2] K. Bansal, A. Reynolds, C. W. Barrett, C. Tinelli. *A New Decision Procedure for Finite Sets and Cardinality Constraints in SMT*. IJCAR 2016, LNCS 9706, pp. 82–98.
- [3] A. Reynolds. *Fix upwards closure for relations*, cvc5 PR #7515, commit 3bcc47a3c, November 2021.